Multi-Agent Ecosystems: Part - 1

Designing Agentic Workflows with CrewAI and LangGraph

Quick Recap

- RAG retrieves data to generate accurate, context-rich answers.
- Agentic RAG plans and acts across steps, enabling smart automation.
- AutoGPT and Anthropic agents power multi-step, goal-driven tasks.
- Agentic systems help PMs with adaptive, real-time decision support.
- Compared to traditional tools, agentic approaches are faster and smarter.

Engage and Think

Imagine you are leading a major product launch involving design, engineering, marketing, and legal teams. Each team uses specialized tools, follows different workflows, and operates on overlapping timelines. The success depends on seamless coordination, timely handoffs, and clear ownership across functions.

You decide to use CrewAI to coordinate the workflow. You can either rely on a single agent or build a multi-agent system where each agent represents a team with its own role and tools.

How would you design this CrewAI system to reflect how your teams work and deliver together?

Learning Objectives

By the end of this lesson, you will be able to:

- Explain how agent-based systems function and where they apply in product workflows
- Configure a basic multi-agent setup using CrewAI with defined roles and tools
- Execute and monitor communication in a two-agent system using CrewAI
- Analyze coordination strategies and evaluate latency, accuracy, and fallback trade-offs
- Assess the strengths of LangGraph and CrewAI for agent-based product design

CrewAI: Overview

CrewAI Framework: Overview

It is a lightweight, Python-based framework that creates multi-agent systems where each agent is assigned a specific role, goal, and set of tools.

It is designed to simulate real-world team collaboration using autonomous AI agents that can coordinate, communicate, and execute tasks intelligently.

CrewAI Framework: Benefits

It helps to automate team-like collaboration across workflows. The benefits are as follows:

Models cross-functional teams with autonomous agents

Supports single-agent and multi-agent setups

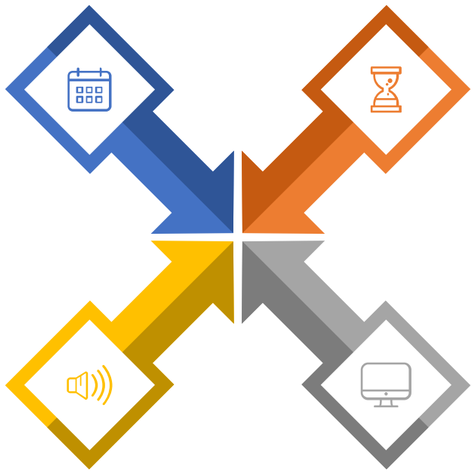
Automates workflows like onboarding, research, and campaigns

Integrates with LangChain, OpenAI, and custom APIs to enhance task execution and decision-making

CrewAI Framework: Core Features

The framework empowers teams to work in harmony, ensuring each element contributes to the overall objective.

Defines roles with specific goals and permissions



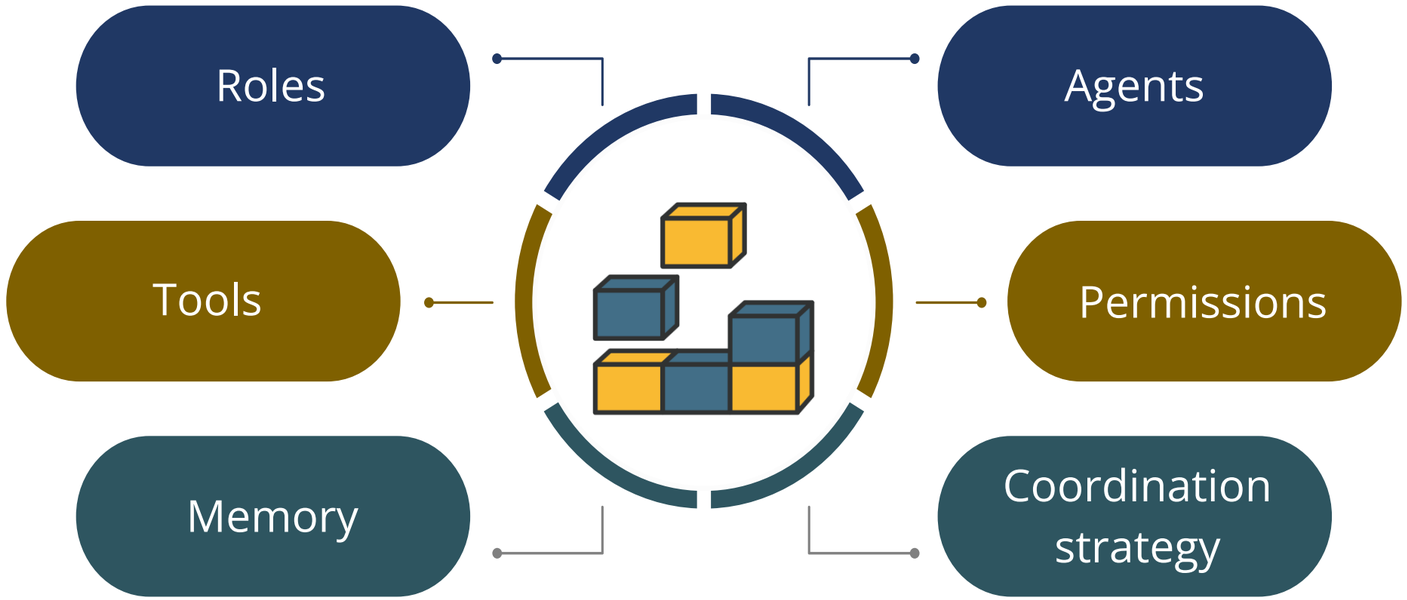
Coordinates agents using serial, parallel, or hierarchical flows

Integrates tools for contextual task execution

Manages memory through sharing or isolation between agents

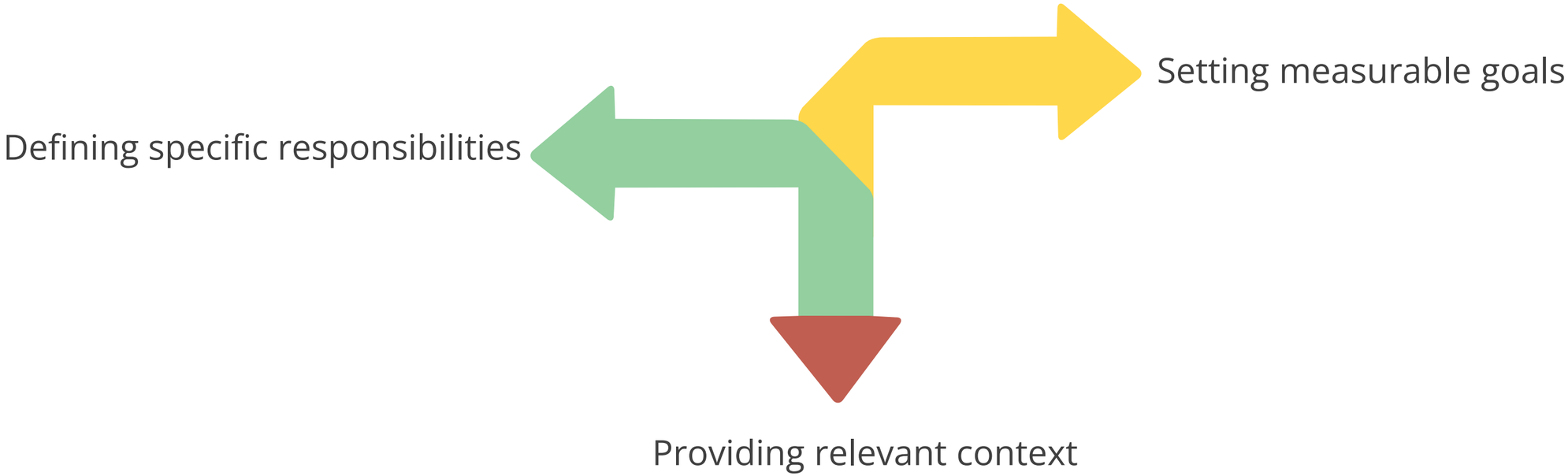
Building Blocks of CrewAI

CrewAI is structured around modular components that work together to model intelligent team-based workflows. Below are the key components of the system:



Building Blocks of CrewAI: Roles

It defines the purpose, goals, and context of an agent within the CrewAI system. Key tasks of roles include:



Purpose

They ensure each agent has a clear responsibility and objective within the workflow.

Building Blocks of CrewAI: Agents

They are autonomous entities assigned to perform tasks based on their defined roles. Key tasks of agents include:

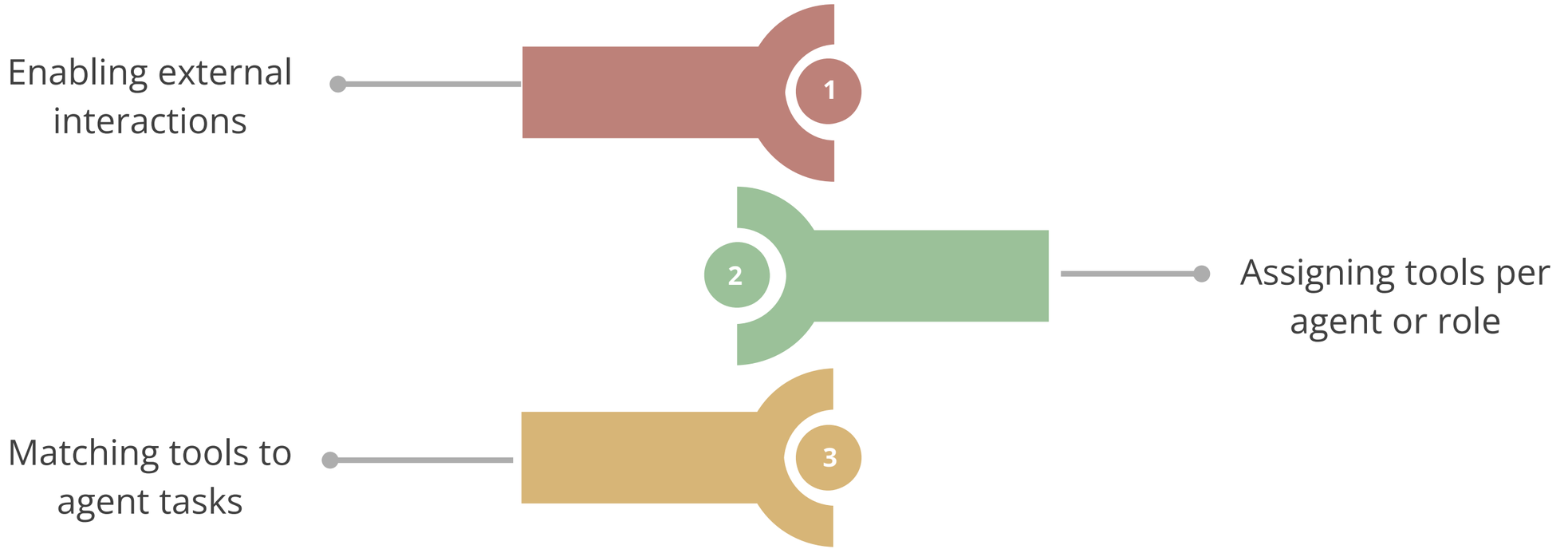
- Following assigned roles
- Communicating with other agents
- Triggering actions independently

Purpose

They execute tasks, make decisions, and interact with tools or other agents within the system.

Building Blocks of CrewAI: Tools

They are functional modules that give agents the ability to perform specific actions. Key tasks of tools include:

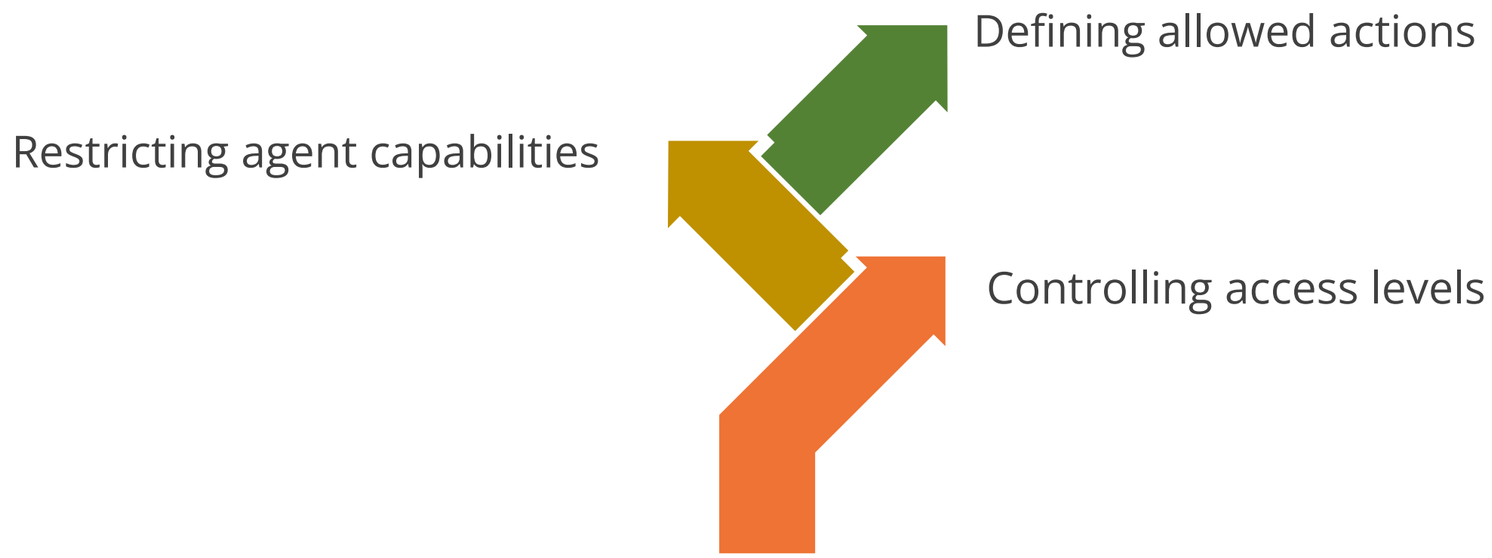


Purpose

They enable agents to interact with external systems, retrieve data, or perform computations.

Building Blocks of CrewAI: Permissions

They act as safeguards, guiding agents to operate within authorized parameters. Key tasks of permissions include:



Purpose

They prevent agents from exceeding their role boundaries.

Building Blocks of CrewAI: Memory

It defines how agents store, access, and share information across tasks. Key tasks of memory include:

Purpose

It allows agents to reference past interactions or operate with isolated knowledge when needed.

Building Blocks of CrewAI: Coordination Strategy

It defines how multiple agents operate together within a CrewAI system to complete tasks efficiently and accurately. Key tasks of the coordination strategy include:

Executing tasks in sequence

Running agents in parallel

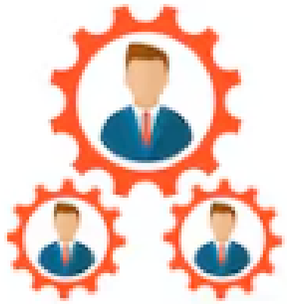
Organizing agents hierarchically

Purpose

It ensures agents collaborate in a way that reflects team dynamics, reduces delays, and adapts to task complexity.

Role Goals in CrewAI

It defines the specific objectives an agent is expected to achieve based on its assigned role within a CrewAI system. The key tasks of role goals are as follows:



Aligning objectives with team functions

Setting task-specific outcomes

Avoiding duplication across roles

How to Define Role Goals in CrewAI

The following are the points to consider when defining role goals:

Identify the role’s purpose based on the task it supports within the larger workflow

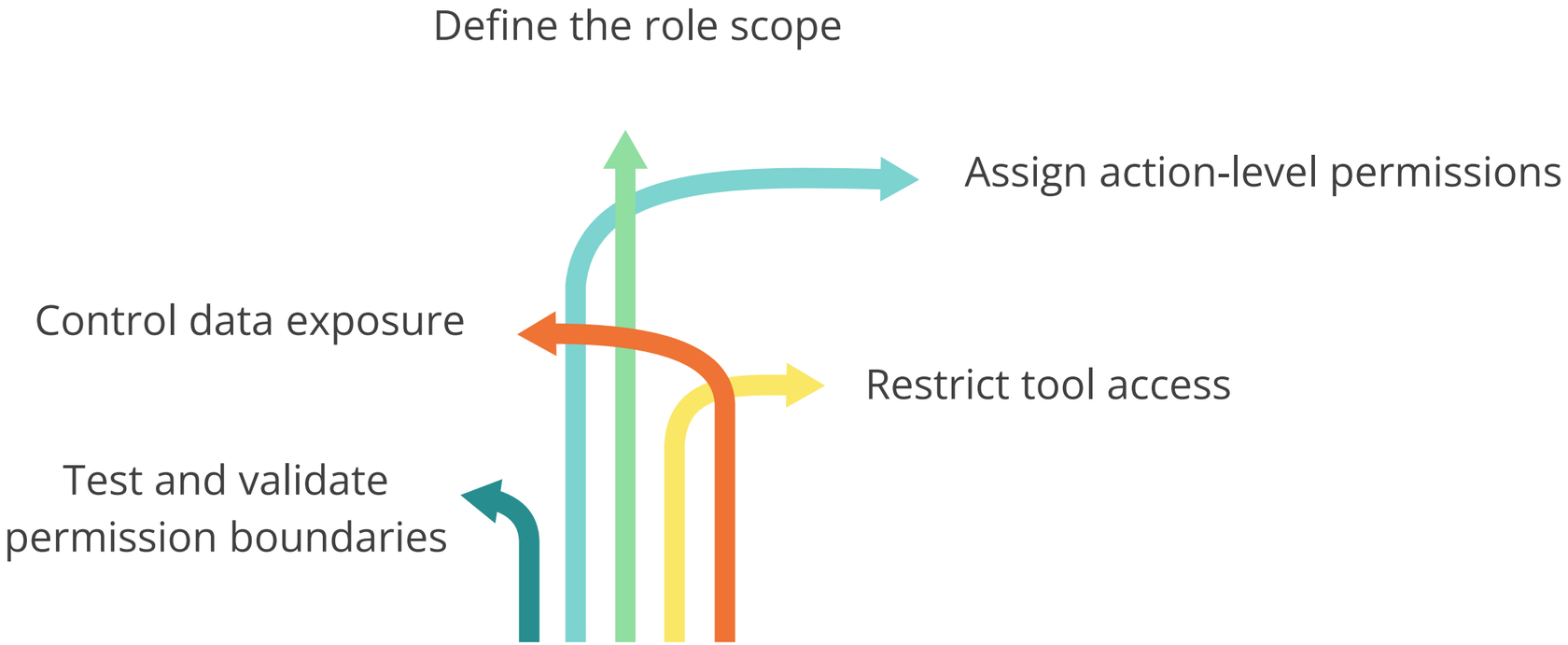
Specify what success looks like by writing a clear, outcome-oriented goal

Limit the scope of responsibility to avoid overlaps with other roles

Choose supporting tools and permissions that match the goal

How to Set Agent Permissions

The following are the points to consider when setting agent permissions:



How to Assign Tools to Roles

The following points are essential when mapping tools to roles:

Understanding the role’s task

Mapping each tool to a specific function

Avoiding tool overload by limiting each role

Ensuring compatibility between the tool and the agent's expected input or output

Reviewing access levels to confirm the agent has the proper permissions

Memory Isolation: Overview

It is a configuration where each agent retains its own private memory, inaccessible to others, ensuring independent processing and decision-making. It can be used in the following scenarios:

- It supports privacy compliance in fields like financial services and healthcare.
- It suits agents with unrelated goals, such as fraud detection and marketing automation.

Benefit

It improves data privacy, security, and modularity—ideal for tasks that require confidentiality or parallel execution without cross-contamination.

Memory Sharing: Overview

It is a configuration where multiple agents access and write to a shared memory space, enabling them to exchange knowledge, state, or context during task execution. It can be used in the following scenarios:

- It fits team-based workflows or tightly coupled task systems.
- It improves efficiency in transparent environments, such as co-writing tools and real-time collaboration platforms.

Benefit

It enhances coordination, enabling agents to make context-aware decisions and adapt to each other’s actions dynamically.

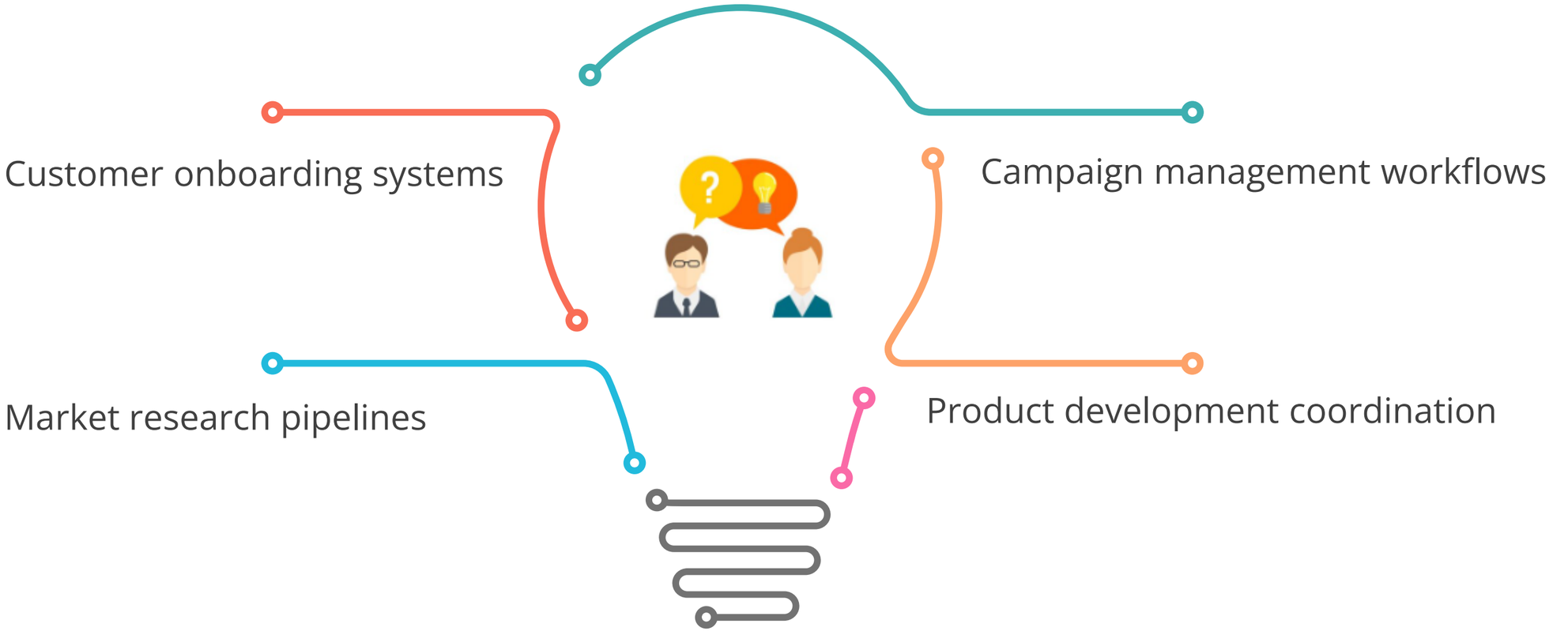
Memory Isolation vs. Memory Sharing

The key differences between memory sharing and isolation are outlined as follows:

Memory isolation	Memory sharing
Supports autonomy and task separation	Encourages coordination and joint decision-making
Prevents unintended influence or data exposure	Promotes real-time visibility into other agents actions
Ideal for tasks requiring confidentiality or operational independence	Ideal for tasks requiring collaboration or synchronized workflows
Strengthens data privacy and modularity	Enhances context-awareness and coordination
Limits shared learning and may require extra coordination	Increases complexity due to inter-agent dependencies

Building Intelligent Workflows with CrewAI

The common scenarios where CrewAI excels include:



CrewAI: Use Case

Optimizing agent workflows with CrewAI to streamline loan approval and risk assessment in financial services

Scenario: A product manager at a digital banking platform must accelerate loan processing while ensuring compliance with strict regulatory requirements. Tasks such as applicant identity verification, credit scoring, fraud detection, loan eligibility analysis, and compliance documentation are distributed across specialized AI agents.

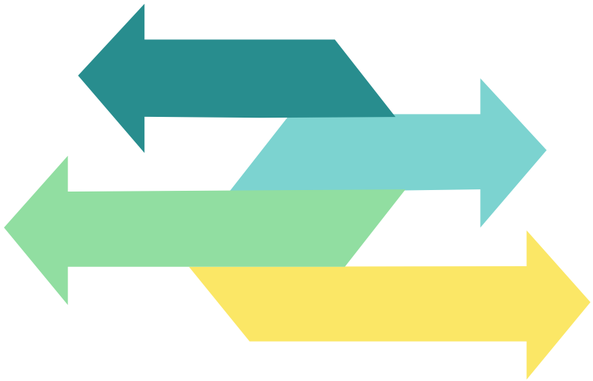
CrewAI: Use Case

Solution: The product manager uses CrewAI to link specialized AI agents for tasks such as identity verification, credit scoring, fraud detection, eligibility checks, and compliance documentation.

CrewAI does the following:

Performs each task accurately and moves them forward

Reduces approval times



Maintains compliance while processing more loan applications

Minimizes errors in the workflow

Quick Check

Which of the following best describes the purpose of assigning tools to roles in a CrewAI system?

A. To allow agents to perform more tasks without restrictions

B. To give agents access to every tool for greater flexibility

C. To ensure agents use only the tools required for their role

D. To let agents override the tool access assigned to other roles

Demo: Building a CrewAI-Powered Agent with OpenAI

Duration: 30 minutes

Scenario: A product manager at a digital knowledge platform needs to manage an increasing volume of user queries, many of which require real-time updates and dynamic information. At present, these queries are reviewed and answered manually, leading to delays, inconsistencies, and rising operational costs. The goal is to automate the query-handling process using a multi-agent system that can identify when real-time data retrieval is needed and provide users with fast, accurate, and context-aware responses while maintaining efficiency and cost control.

DEMONSTRATION

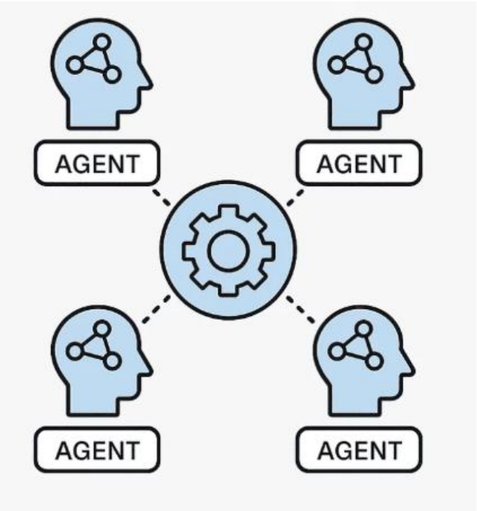
Note:

Please download the demo document from the reference material section to perform step-by-step execution.

Design Patterns in Agent Communication

Agent Communication: Overview

It refers to how autonomous agents exchange information, coordinate tasks, and make decisions either collaboratively or independently.



Why it matters

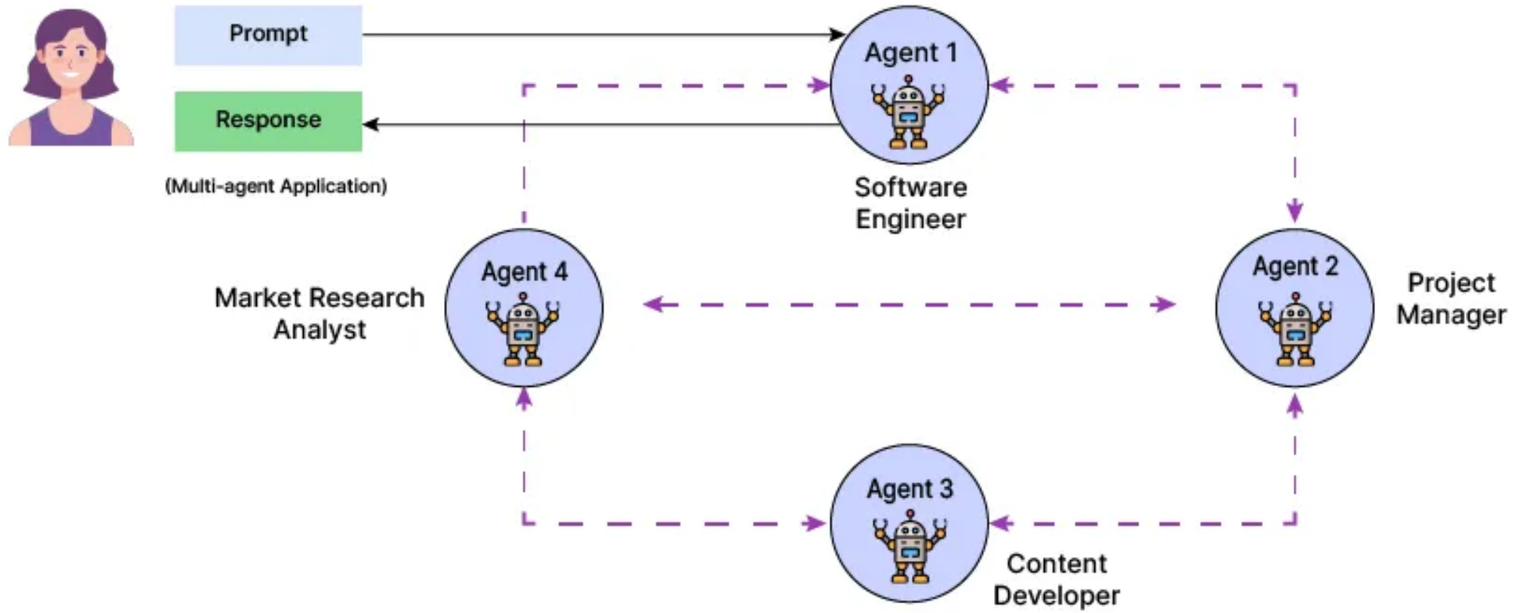
It enables task coordination, supports modular and scalable design, and reduces complexity in distributed workflows.

Core Elements of Agent Communication

Strong communication between agents is built on a foundation of the following core elements:

Agent Coordination Patterns: Overview

They describe how autonomous agents organize and manage actions to achieve system-level goals. The following diagram illustrates multi-agent communication and coordination:



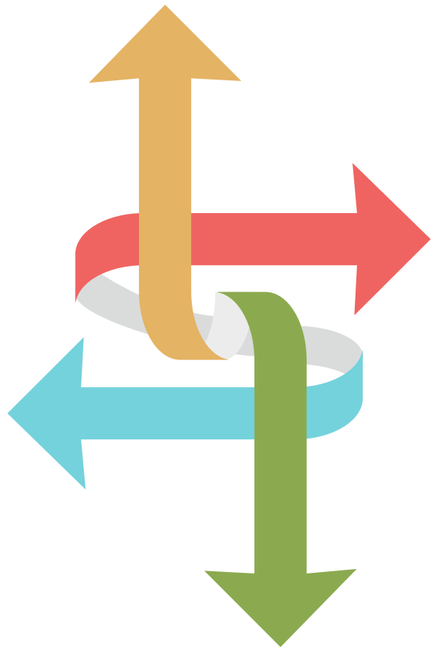
Why it matters

It boosts efficiency and accuracy while enabling flexible agent design. It also ensures proper sequencing and alignment with system goals.

Types of Agent Coordination Patterns

The following are common coordination patterns that define how agents organize and execute tasks within a system:

Serial coordination



Parallel coordination

Hierarchical coordination

Dynamic and adaptive coordination

Serial Coordination

This pattern involves agents completing tasks in a fixed order, where each step depends on the completion of the previous one. Key features include:

- Sequential execution
- Task dependency
- Simplicity and clarity
- Deterministic behavior

This approach enables orderly workflows, predictable outcomes, and controlled information flow.

Parallel Coordination

This pattern enables multiple agents to operate independently and simultaneously. Key features include:

Concurrent execution

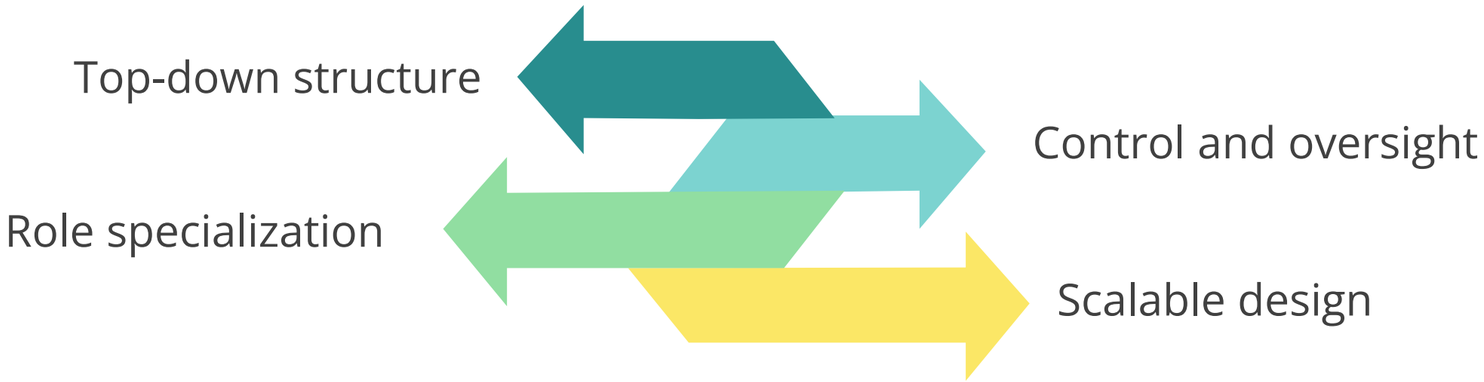
Post-execution merging

No immediate dependencies

Improved efficiency

Hierarchical Coordination

In this pattern, agents follow a parent–child structure where higher-level agents control or delegate tasks to lower-level agents. Key features include:

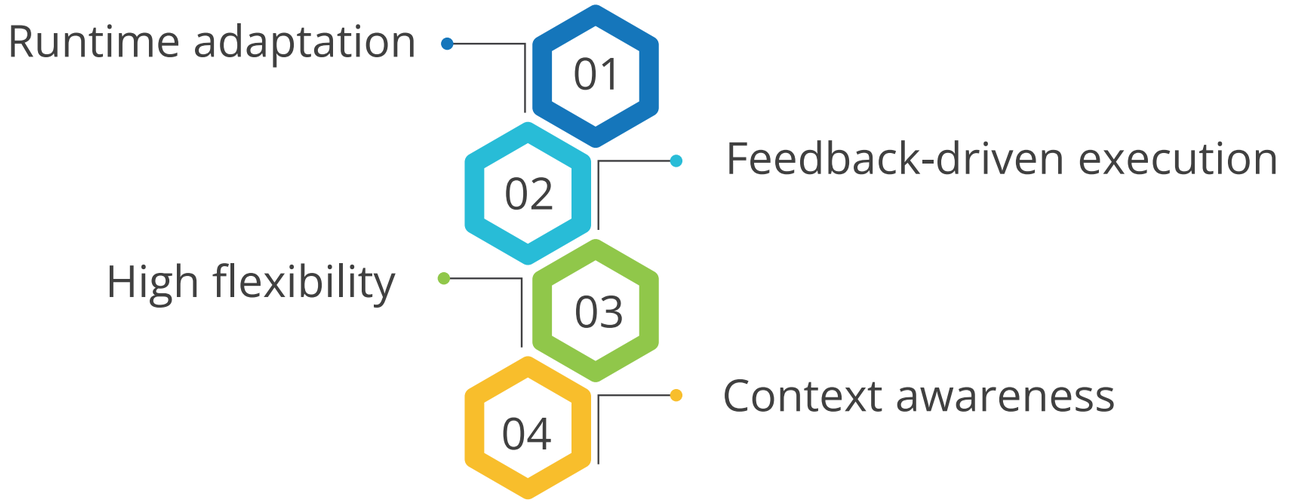


Why it matters

It supports structured delegation, enables layered control, and improves clarity in complex agent hierarchies.

Dynamic and Adaptive Coordination

In this pattern, agents adjust their behavior at runtime in response to feedback, context, or evolving goals. Key features include:

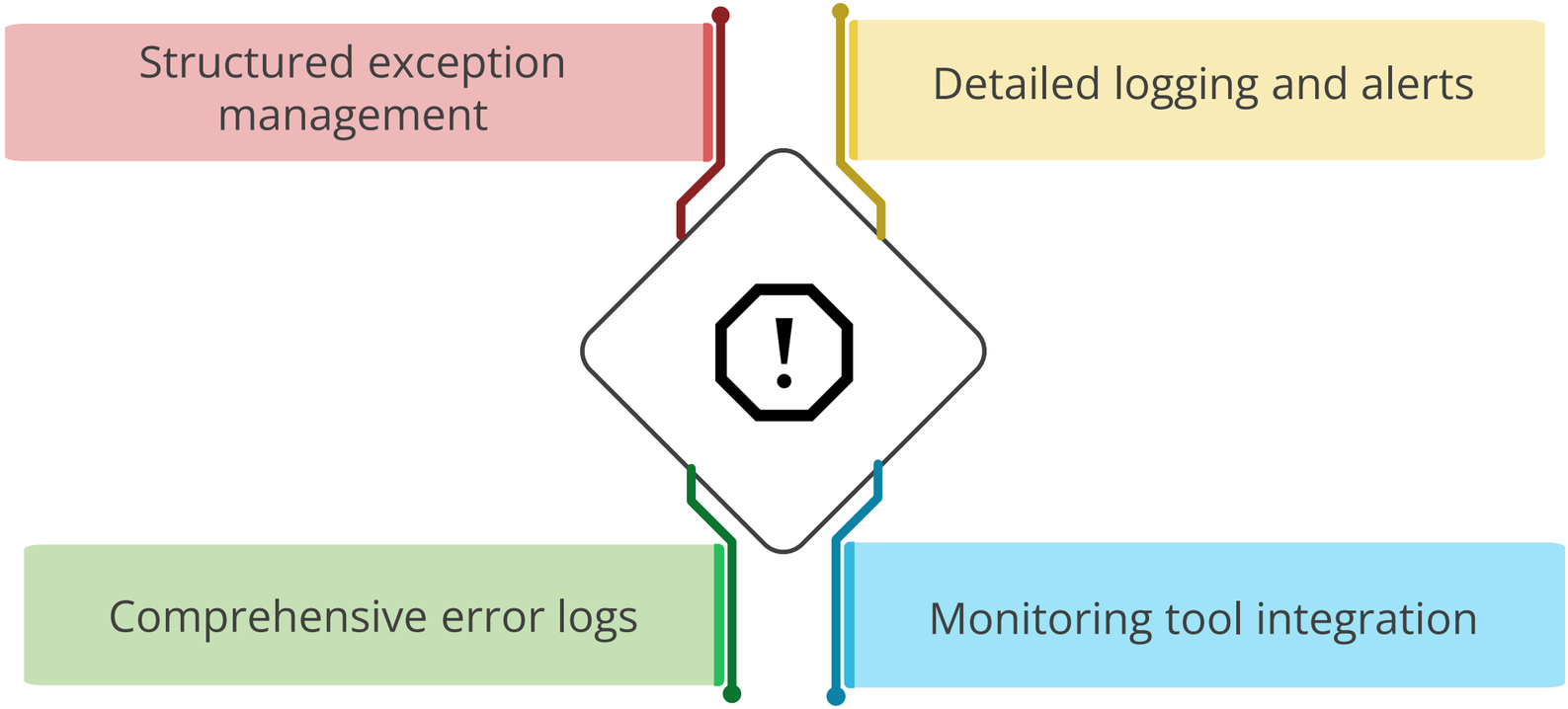


Why it matters

This pattern supports real-time responsiveness and robustness in unpredictable environments. It enables agents to adapt to new inputs without predefined instructions.

Managing Agent Failures

CrewAI supports efficient detection and recovery from agent failures. Key factors that help address agent failures are as follows:

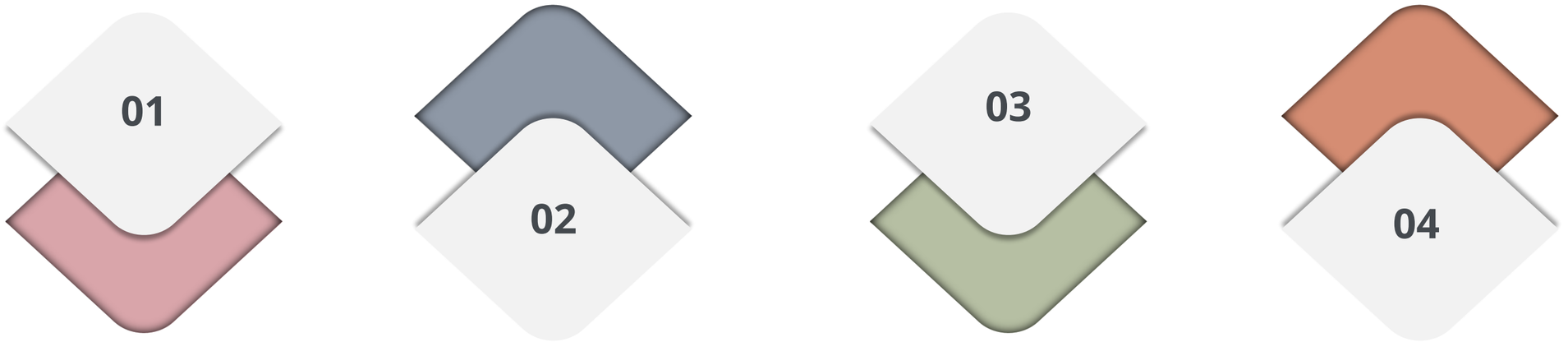


Fallback Mechanisms in CrewAI

CrewAI offers multiple fallback strategies to ensure continuity and resilience in agent workflows. The key capabilities that make these fallback strategies effective are as follows:

CrewAI Guardrails: Overview

CrewAI includes built-in guardrails to validate agent outputs before advancing to the next step. These guardrails perform the following key functions:



Validates output

Corrects loop automatically

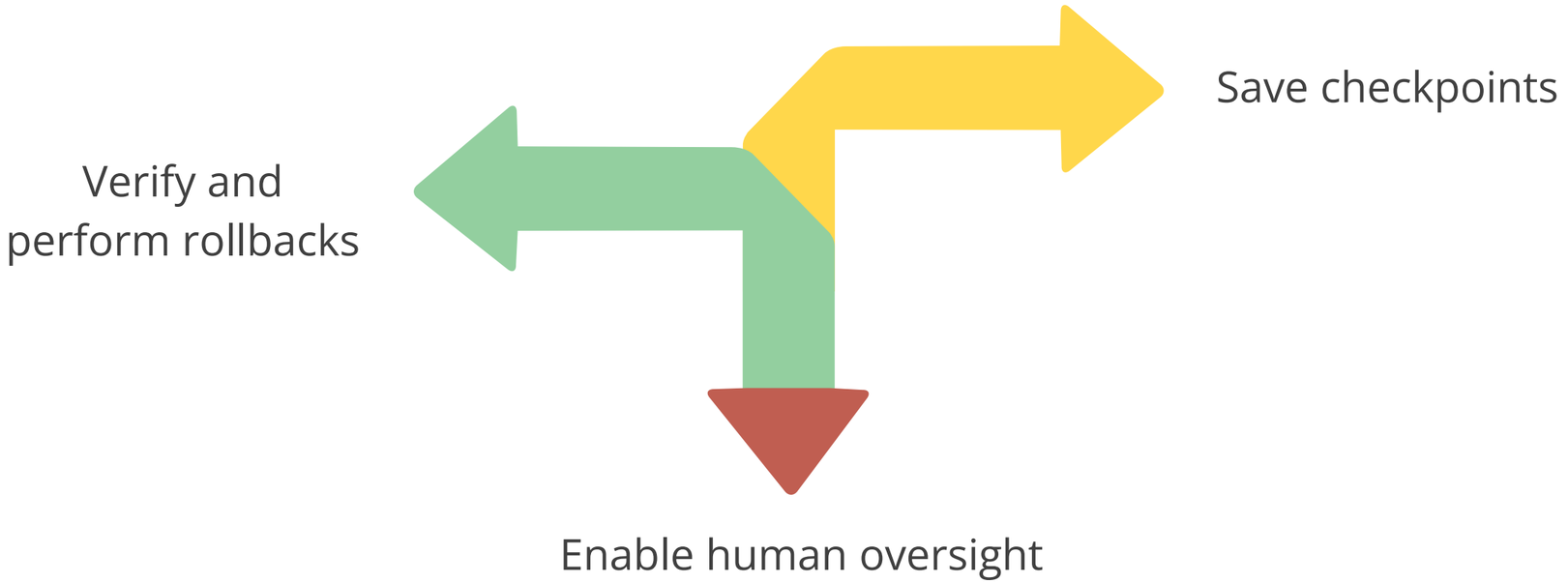
Retries configuration

Ensures integrity

This helps to ensure reliability and correctness in multi-agent workflows.

Architectural Considerations in CrewAI

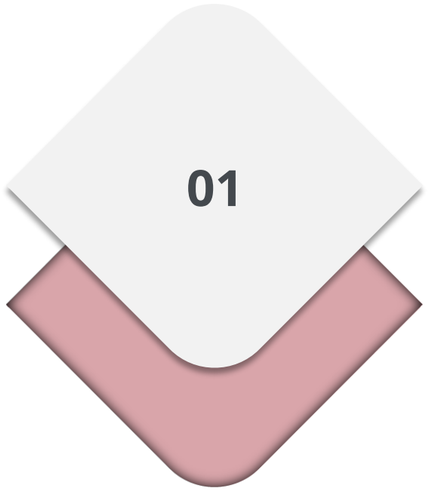
CrewAI supports a resilient and observable system design through key architectural practices. The key design considerations are as follows:



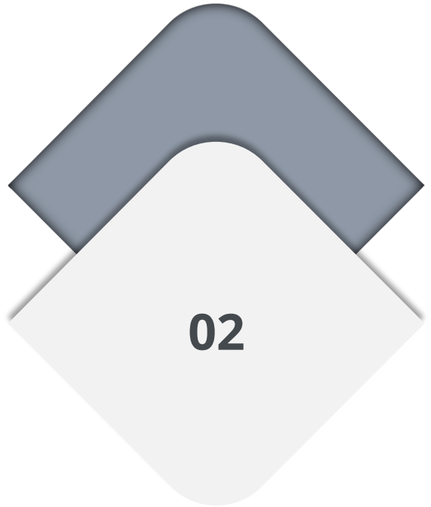
These checks enhance fault tolerance, recovery, and human oversight.

Key Performance Trade-Offs in CrewAI

Designing multi-agent systems using CrewAI requires thoughtful consideration of performance trade-offs. Two of the most critical factors to balance are:



Latency



Accuracy

Key Performance Trade-Offs: Latency

It refers to the time it takes for a system to process a request and generate a response.
The impacts of high latency are as follows:

Diminishes user experience

Impairs real-time decision-making

Reduces system responsiveness

Causes financial losses in time-sensitive environments

Key Performance Trade-Offs: Accuracy

It refers to how correct and precise a system’s output is. The impacts of low accuracy are as follows:

Produces unreliable or misleading results

Reduces user trust and confidence

Leads to serious errors like misdiagnoses or financial mistakes

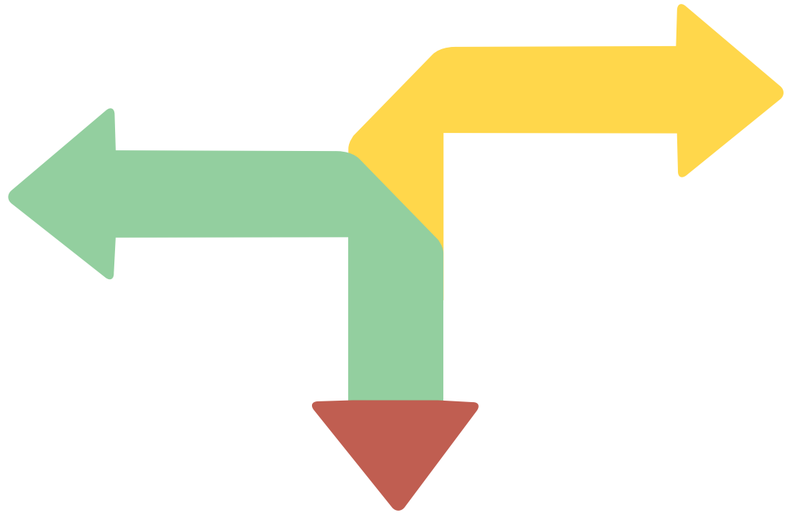
Balancing Latency and Accuracy

Achieving low latency while maintaining high accuracy is often difficult and requires careful trade-offs. The key design considerations for managing latency and accuracy are as follows:

- Optimizing agent communication to reduce delays
- Selecting models that balance speed and precision
- Configuring tasks to minimize processing overhead

Optimization Strategies in CrewAI

Key strategies to balance latency and accuracy in CrewAI are as follows:



Optimize models and algorithms

Enhance architecture and infrastructure

Apply CrewAI specific design strategies

Optimization: Use Case

Optimizing agent workflows with CrewAI for scalable product operations

Scenario: A product manager at an AI-powered productivity platform must scale customer onboarding without raising operational costs. Tasks such as user verification, account setup, feature recommendation, and training are distributed across specialized AI agents.

Optimization: Use Case

Solution: The product manager integrates CrewAI to optimize coordination and execution across agents. CrewAI enables flexible agent collaboration using tailored coordination strategies such as:

Serial coordination for linear setup flows

Parallel coordination for tasks like training and feature activation

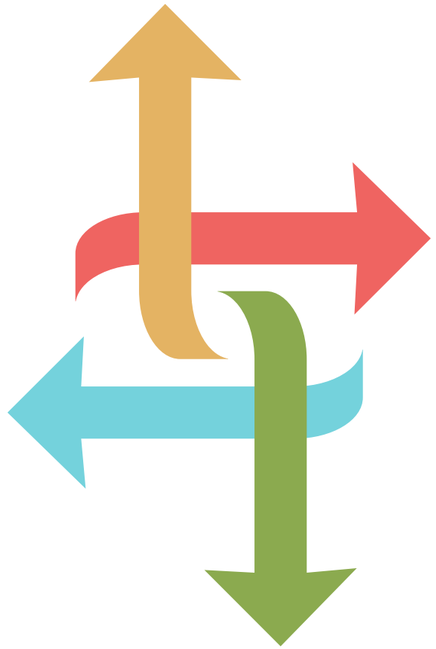
Dynamic coordination to switch paths based on user responses or delays

Hierarchical coordination to escalate complex cases to supervisor agents

Optimization: Use Case

This CrewAI-powered system applies the following:

Guardrails help verify outputs
before task transitions



Architectural controls fine-tune
performance and avoid bottlenecks

Fallback mechanisms reroute
failed tasks dynamically

Latency and accuracy balancing to optimize
both speed and precision based on user tier

Quick Check

Which of the following coordination patterns involves agents working independently at the same time without waiting for each other’s output?

A. Serial coordination

B. Hierarchical coordination

C. Parallel coordination

D. Adaptive coordination

Demo: Executing Serial and Parallel Workflows Using CrewAI Agents

Duration: 30 minutes

Scenario: A product leader at a global FMCG company needs timely insights on how AI is transforming product development, consumer engagement, and supply chain efficiency. Gathering and summarizing this information manually is time-consuming and delays product strategy decisions. The challenge is to automate this workflow using AI agents—one to research the latest AI trends in FMCG and another to generate executive-ready summaries—so the product team can make faster, data-informed decisions in a highly competitive market.

Note:

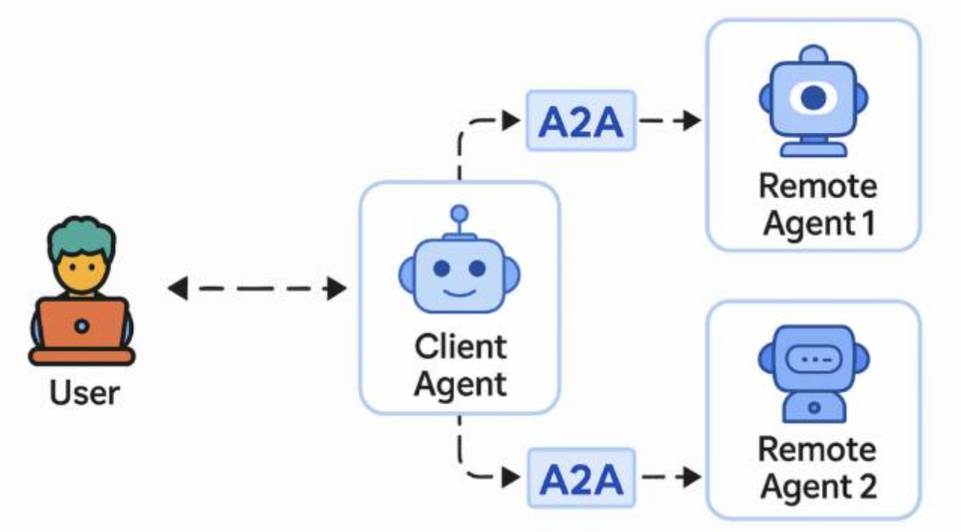
Please download the demo document from the reference material section to perform step-by-step execution.

DEMONSTRATION

Advanced Agent Communication Using Google A2A Protocol

Google A2A Protocol: Overview

It is a standard communication protocol designed to enable autonomous AI agents to collaborate across frameworks, platforms, and languages.

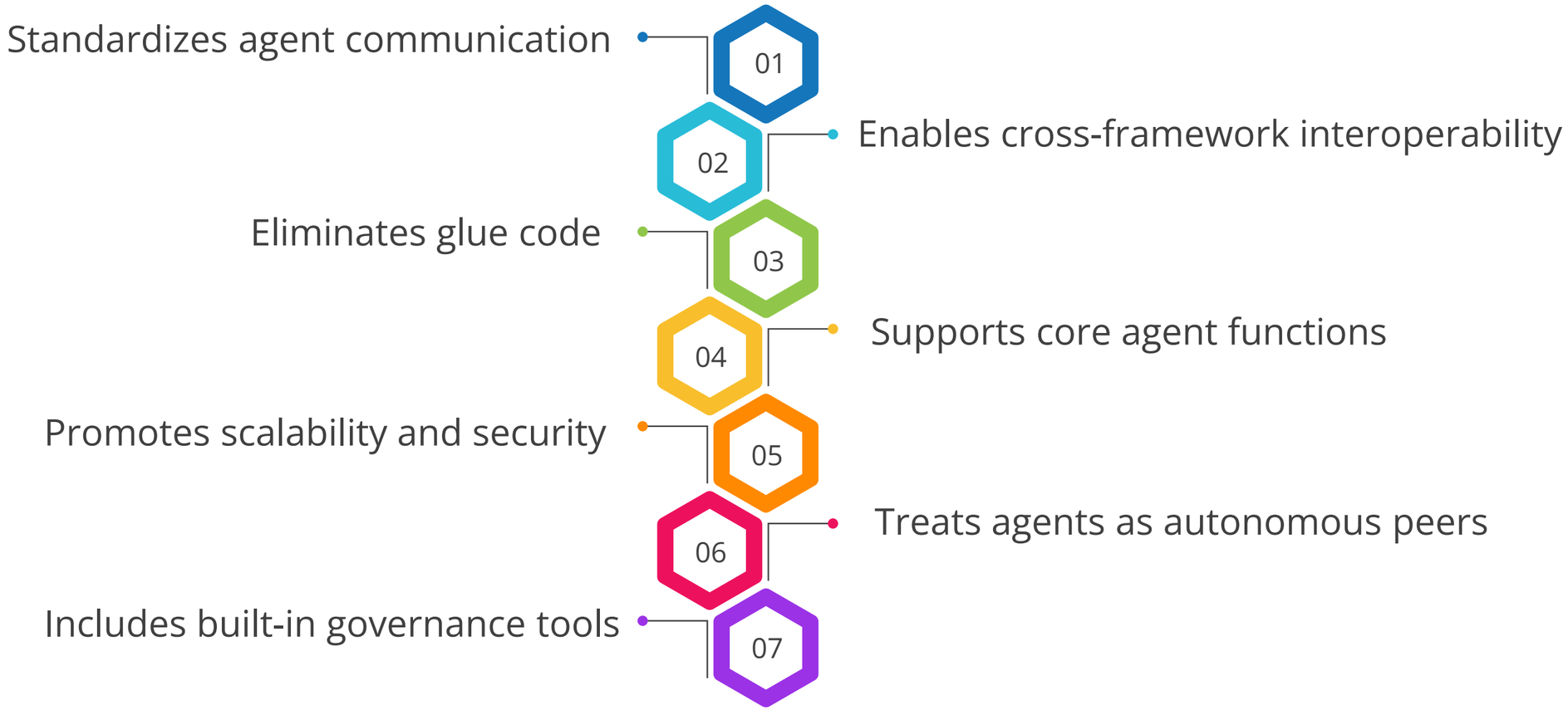


Why it matters

A2A solves agent isolation by standardizing communication across proprietary frameworks, tools, and languages.

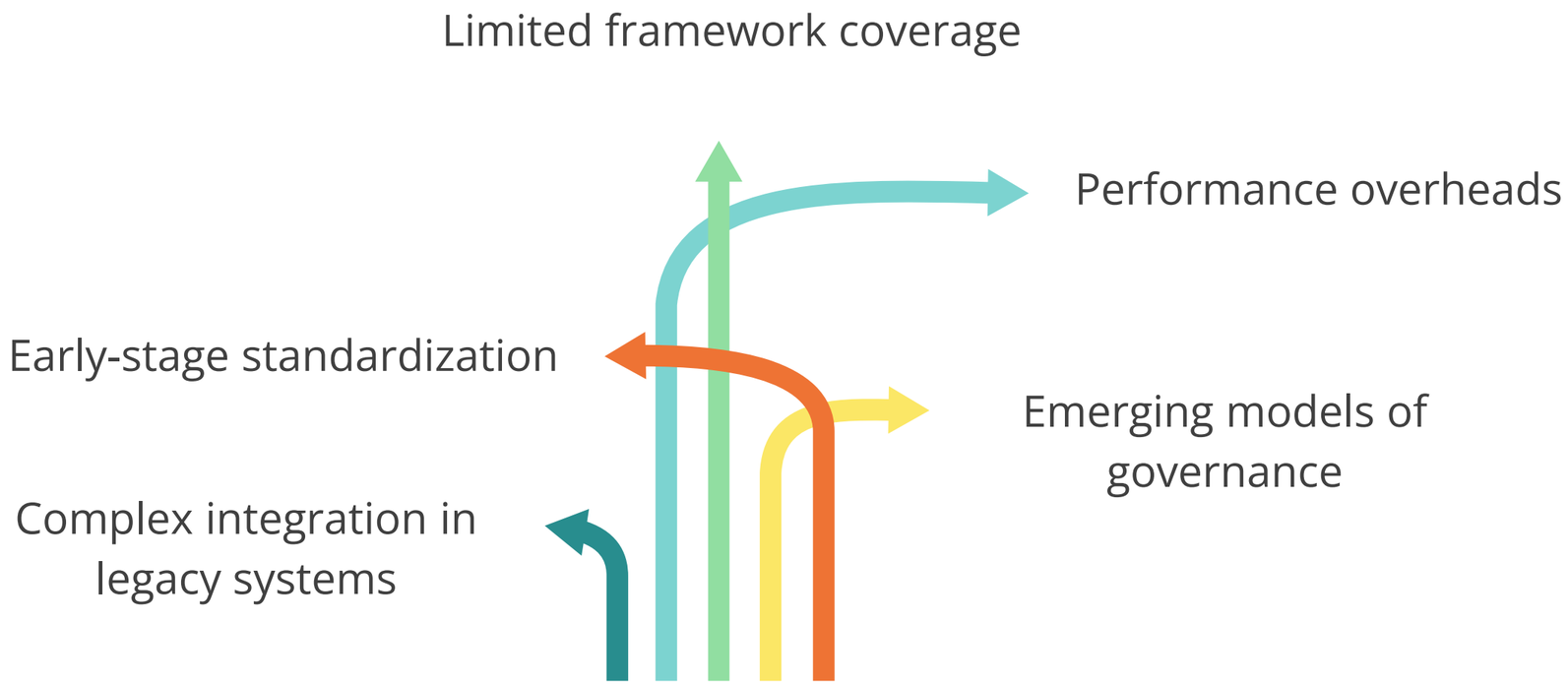
Google A2A Protocol: Features

The key features that make A2A a powerful and flexible agent communication protocol are as follows:



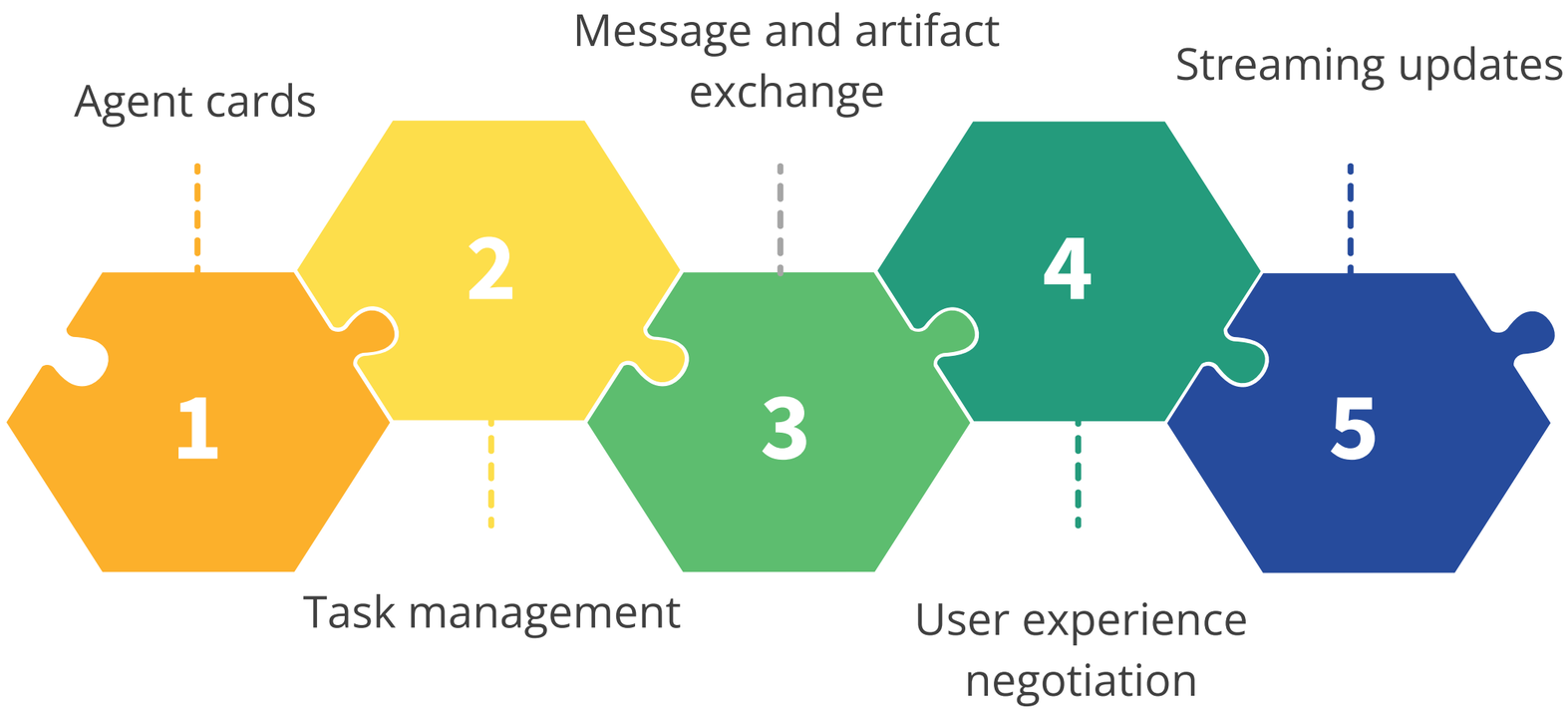
Google A2A Protocol: Limitations

The key limitations of the Google A2A Protocol are as follows:



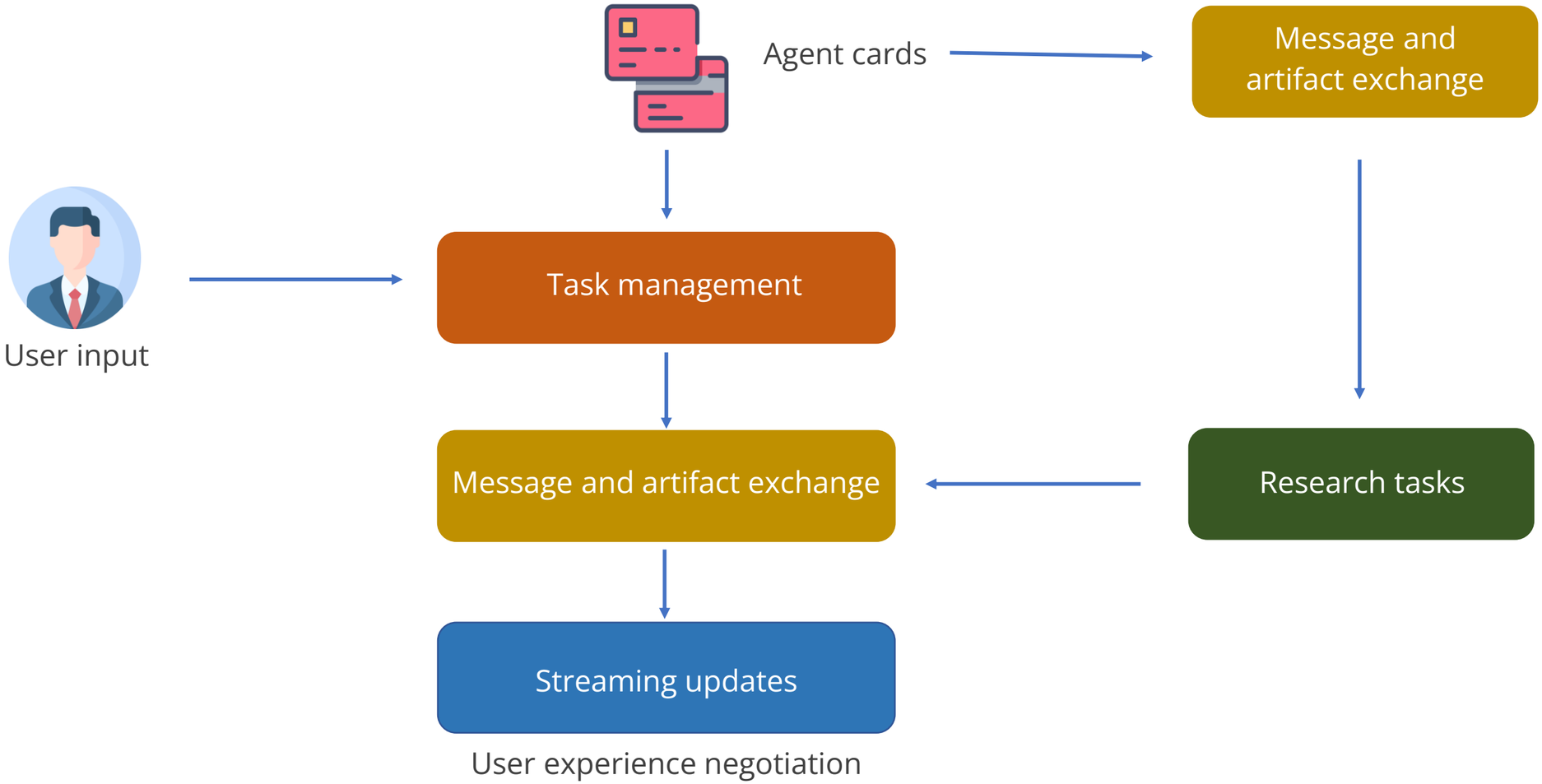
Google A2A Protocol: Core Components

The following are the key components that define how agents interact and collaborate using the A2A protocol:



Google A2A Protocol Core Components: Example

The following example shows how the A2A protocol components enable seamless collaboration between AI agents in a product research assistant scenario:



Design Features for Agent Communication and Coordination

The core capabilities and design features that enable structured, secure agent collaboration are as follows:

Agent card	Provides essential metadata, including ID, API endpoints, capabilities, and permissions
Structured task messages	Uses a standard schema to define task goals, inputs or outputs, execution rules, and required permissions
Task lifecycle support	Manages short- and long-duration tasks from submission to completion or failure

Design Features for Agent Communication and Coordination

The core capabilities and design features that enable structured, secure agent collaboration are as follows:

Multimodal exchange	Supports multiple formats like text, JSON, images, video, and form-based inputs
Security by design	Implements OAuth 2.0, JWT, RBAC, digital signatures, and encryption to ensure secure agent communication
Schema design	Promotes standardization, traceability, and compliance with enterprise requirements

A2A Across Industries: Real-World Applications

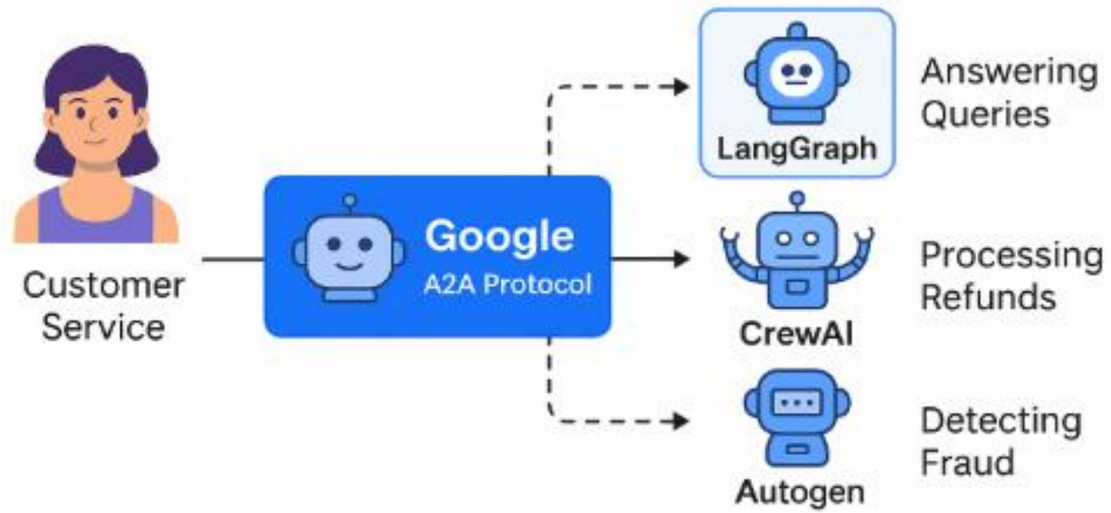
A2A delivers transformative value across industries. Some of these industries are as follows:

Finance	Supports agent collaboration for fraud detection, trading, and risk analysis
Healthcare	Coordinates diagnostics, treatment planning, and patient data handling
Supply chain and logistics	Manages inventory, tracking shipments, and planning logistics
Customer service	Handles inquiries, retrieves information, and manages escalations

Google A2A Protocol: Use Case

Seamless collaboration across heterogeneous agent frameworks

Scenario: A global enterprise is building an AI-driven customer service platform where different teams have developed agents using CrewAI. These agents manage tasks like answering queries, processing refunds, and detecting fraud. However, the agents can't communicate directly due to incompatible interfaces, creating fragmentation and duplication of work.



Google A2A Protocol: Use Case

Solution: The organization adopts the Google A2A Protocol to standardize agent-to-agent communication across frameworks, enabling a unified multi-agent system without rewriting existing logic.
With A2A in place, each agent registers using agent cards and communicates through structured task messages. The protocol enables the following:

Discovering and authenticating agents across frameworks

Sharing artifacts and real-time updates via SSE

Negotiating message formats based on recipient capabilities

Delegating tasks like refund processing to the most suitable agent

Ensuring secure, authorized interactions using OAuth2

Quick Check

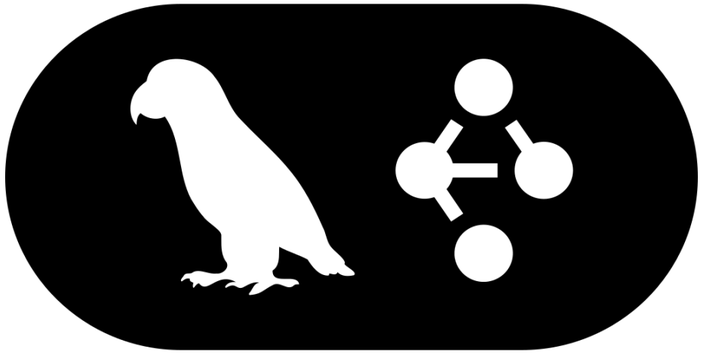
What is the primary goal of the Google A2A Protocol?

A. Enables agent data storage in distributed systems
B. Standardizes agent communication and coordination
C. Generates training data for AI models
D. Replaces human decisions in enterprise tools

Agents with LangGraph: Overview

LangGraph: Overview

It is a framework for building stateful, multi-agent workflows using graph-based execution logic.

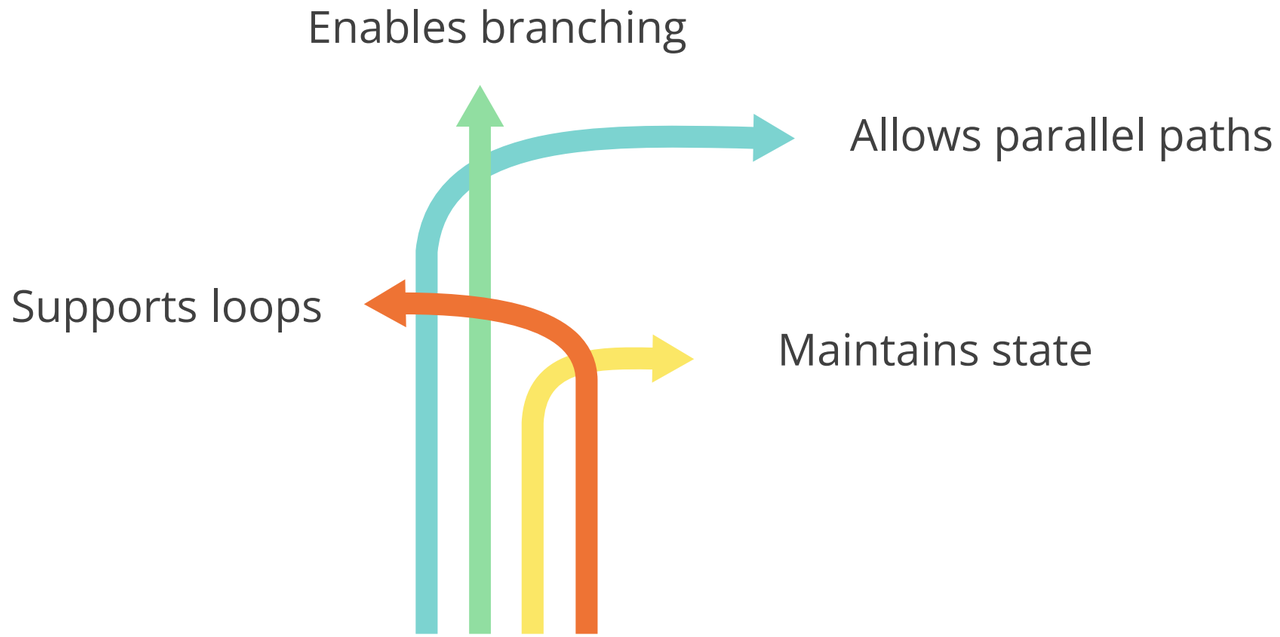


Where is it used?

It is used in applications requiring adaptive agent workflows, such as chatbots, multi-agent systems, and tool-integrated AI pipelines.

LangGraph Graph Orchestration: Overview

LangGraph uses a graph structure to define how agents, tools, or functions connect and interact during workflow execution. The benefits are as follows:



Instead of a fixed, step-by-step script, LangGraph builds a flexible execution map that adapts based on outcomes.

LangGraph vs. LangChain

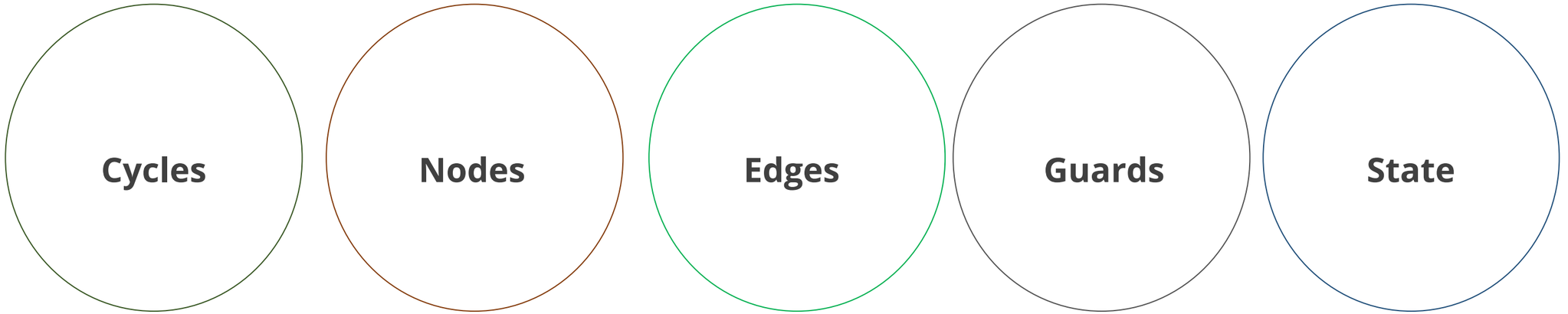
Aspect	LangGraph	LangChain
Purpose	Build multi-agent, stateful workflows using a graph-based execution model	Build applications powered by LLMs using modular components such as chains, prompts, and memory
Workflow structure	Organize execution into interconnected nodes or agents within a graph	Organize execution into sequential or branching chains
Ideal use case	Use for complex, adaptive multi-agent workflows	Use for single-agent or mostly linear workflows
Execution flow	Allow dynamic, event-driven execution paths that change mid-run	Follow a largely predetermined path with optional branching
State handling	Track state within individual chains or through external memory objects	Maintain a shared, persistent global state across all agents or nodes
Common applications	Apply to chatbots, document question answering, summarization, and retrieval-augmented generation	Apply to multi-agent research assistants, operational AI orchestration, and tool-integrated AI pipelines

Benefits of LangGraph

It extends LangChain by enabling more adaptive, collaborative, and context-aware agent workflows. Key benefits are as follows:

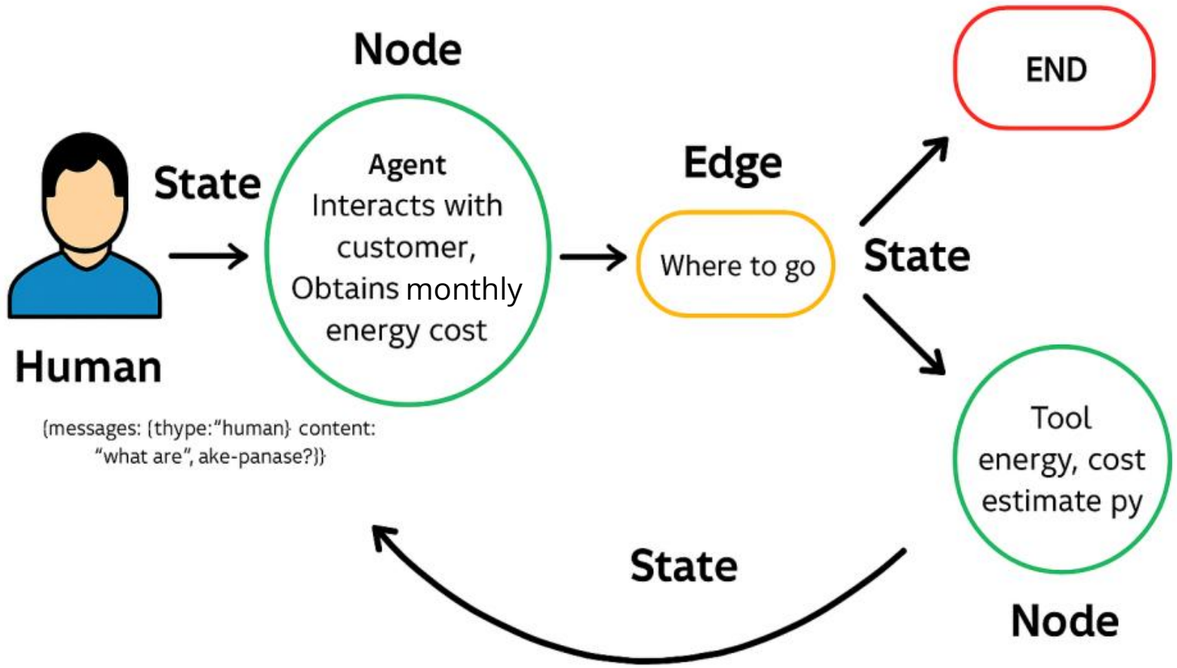
Core Concepts of LangGraph

The core structural elements of LangGraph enable intelligent and adaptive workflows. The elements are as follows:



Core Concepts of LangGraph: Example

The following diagram illustrates how LangGraph executes workflows by combining agent actions, conditional routing, and persistent state flow:



LangGraph: Execution Model

It is a stateful, graph-based system that manages agent workflows using nodes, edges, and conditional logic with persistent context. The benefits are as follows:

Maintains persistent state

It retains memory across nodes to support long-running, context-rich workflows.

Enables pause and resume

It allows workflows to halt at decision points and restart without data loss.

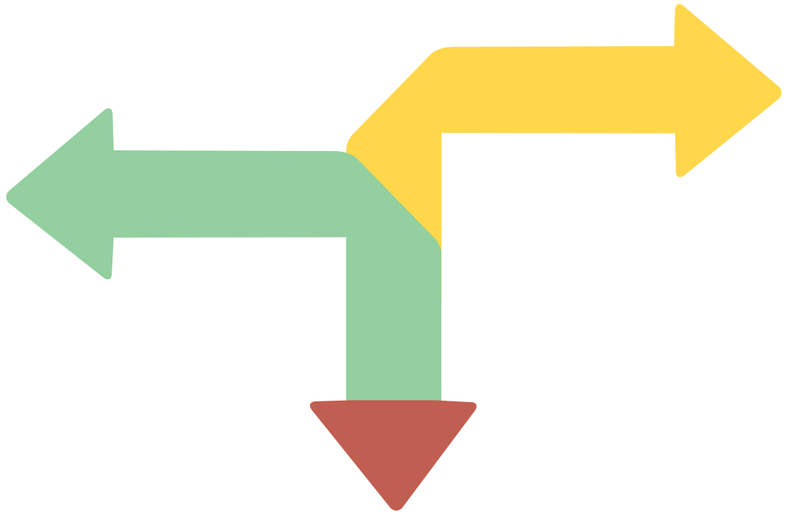
Supports human-in-the-loop

It facilitates workflows requiring manual review or external validation mid-process.

Benefits of LangGraph Integration

The integration of LangChain and LangSmith into LangGraph enhances workflow composition, observability, and scalability in the following ways:

Extends LangChain workflows



Integrates monitoring with LangSmith

Enables modular development

Key Tools for Visual Planning in LangGraph

LangGraph supports the following visual tools that help developers design, debug, and monitor agent workflows more effectively:

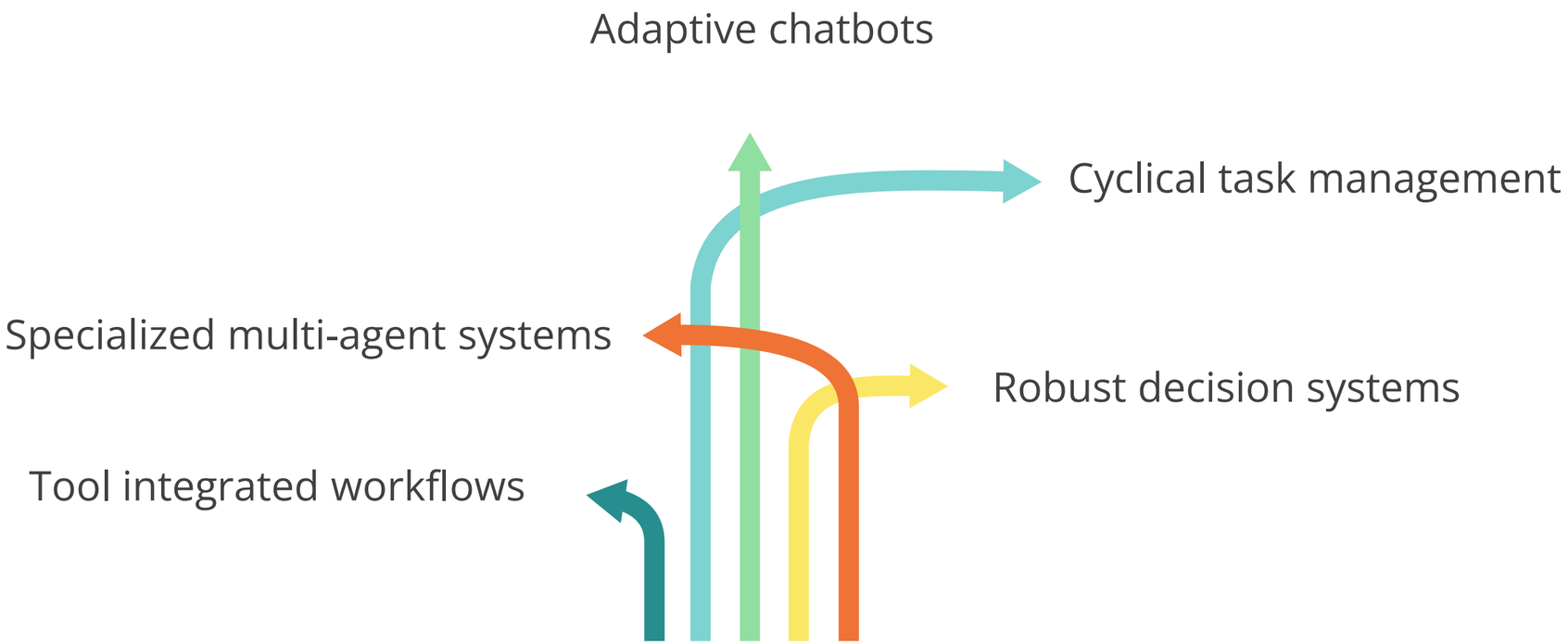
LangGraph Studio

Graphviz

LangSmith

LangGraph: Real-World Applications

It powers advanced agent workflows in real-world scenarios, including:



LangGraph: Use Case

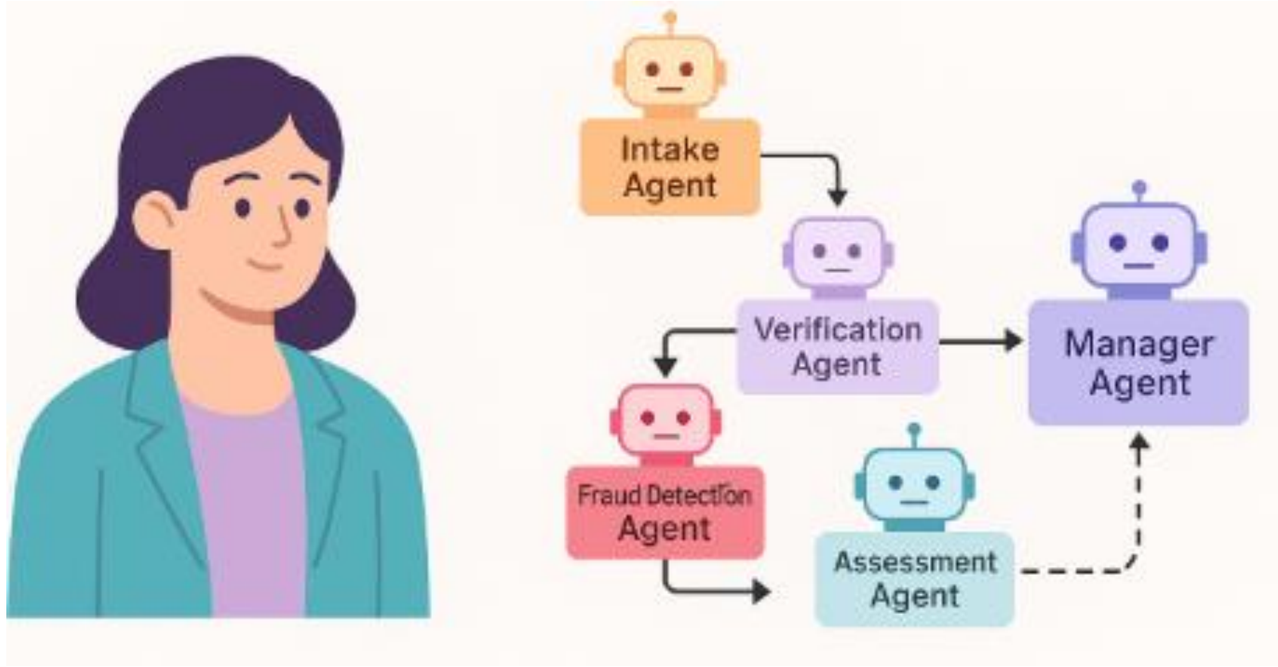
Automating insurance claim processing with LangGraph agents

Scenario: A product manager at an insurance company is responsible for launching a digital claims platform. The process involves multiple teams, including customer service, claims review, fraud detection, document verification, and legal compliance. Previously, all steps were manually coordinated by a central project coordinator, resulting in slow approvals, frequent handoff issues, and limited process visibility.

LangGraph: Use Case

Solution: The product manager decides to implement a LangGraph-powered multi-agent workflow to automate and orchestrate the claim handling process. Each team is represented by a specialized AI agent configured with clear responsibilities and checkpoints.

Using LangGraph, the product manager defines a structured flow where agents perform key roles across the claim lifecycle:



Quick Check

Which of the following best describes a core benefit of using LangGraph in multi-agent systems?

A. It completely replaces the need for LangChain.

B. It enables stateful, graph-based agent orchestration.

C. It removes the need for integrating tools into workflows.

D. It supports only linear execution flows.

Demo: Building a Reactive Agent Workflow with LangGraph and Tavily

Duration: 30 minutes

Scenario: A product lead at a fintech startup wants to monitor global developments in real-time fraud detection methods for UPI systems. The manual process of tracking cybersecurity advancements and regulatory changes is inefficient and prone to oversight. The goal is to automate this workflow using LangGraph, where an agent uses a search tool to gather the latest research and another agent validates and summarizes the findings for internal teams.

DEMONSTRATION

Note:

Please download the demo document from the reference material section to perform step-by-step execution.

Guided Practice

Overview **Duration:30 minutes**

In this guided practice, you will explore how to orchestrate multiple AI agents using a modular, plug-and-play architecture powered by the Agent-to-Agent (A2A) protocol with LangGraph. You will work with agents built using LangChain, CrewAI, and the OpenAI API, all connected into a single, state-aware workflow. This exercise will walk you through how user input is parsed, a research task is triggered using web tools, and a final response is synthesized.

By the end of this session, you will understand how cross-framework agents can collaborate at scale while maintaining shared context, enabling enterprise-ready GenAI workflows.

Key Takeaways

- CrewAI simplifies agent design by structuring roles, tools, memory, and permissions.
- Coordination strategies like serial, parallel, and dynamic improve multi-agent efficiency.
- LangGraph enables stateful workflows with flexible, graph-based agent connections.
- The A2A protocol standardizes agent communication across frameworks and tools.
- Design choices in memory, fallback, and speed-accuracy balance shape system performance.

Q&A